LECTURE NOTES • CAMPUSX

Retrieval-Augmented Generation (RAG)

Complete Notes with Diagrams, Math & Architecture

[YT]	CampusX YouTube Lecture
[Book]	Brown et al. 2020 — Few-Shot Learners
[Tag]	LLMs • Vector DB • Embeddings

Topics: Parametric Knowledge · Fine-Tuning · LoRA · Semantic Search · RAG Pipeline

Contents

1 LLM Parametric Knowledge **2**

2 Three Core Limitations of Standard LLMs **2**

3 Previous Solutions & Their Shortfalls **3**
- 3.1 A. Fine-Tuning [11:00] . . . 3
- 3.2 B. In-Context Learning / Few-Shot Prompting [23:19] . . . 4

4 What is RAG? [34:03] **4**

5 The 4-Step RAG Architecture [37:36] **5**
- 5.1 Step 1 — Indexing (Build the Knowledge Base) [41:16] . . . 5
- 5.2 Step 2 — Retrieval (Semantic Search) [49:13] . . . 6
- 5.3 Step 3 — Augmentation [53:07] . . . 6
- 5.4 Step 4 — Generation [54:06] . . . 7

6 RAG vs Fine-Tuning — Full Comparison [55:36] **8**

7 Beyond Naïve RAG — Advanced Techniques **8**

8 Quick Revision Summary **8**

1 LLM Parametric Knowledge

Definition: Parametric Knowledge

A Large Language Model stores **all its world knowledge** inside its **weights and biases**. This is called *Parametric Knowledge*. The more parameters, the richer and more detailed the stored knowledge.

The standard LLM interaction pipeline is:

$$\underbrace{\text{Query / Prompt}}_{\text{User Input}} \longrightarrow \underbrace{\text{LLM } (\boldsymbol{\theta})}_{\text{Parametric Knowledge}} \longrightarrow \underbrace{\text{Response}}_{\text{Output}}$$

Quality–Parameter Relationship:

$$\text{LLM Quality} \propto \text{Number of Parameters}$$

Example: GPT-3 has $N = 1.75 \times 10^{11}$ parameters. Common sizes: $7B$, $13B$, $70B$ parameters.

Key Insight

More parameters $\Rightarrow$ better pattern memorisation $\Rightarrow$ richer world knowledge,
BUT knowledge is **frozen at training time** and cannot update automatically.

2 Three Core Limitations of Standard LLMs

1. Private Data [06:51]

LLMs train on **public internet** data only. They cannot answer queries about:

- Company documents
- Internal knowledge bases
- Personal PDFs
- Proprietary courses

2. Knowledge Cutoff [07:52]

Training has a **fixed end date**. The model is blind to:

- Recent news & events
- New research papers
- Market changes
- Law / policy updates

3. Hallucinations [09:13]

LLMs are **probabilistic**. When uncertain they may:

- Invent citations
- Fabricate statistics
- State falsehoods confidently
- Make up dates/names

LLM Generation is Probabilistic:

$$\hat{y} = \arg\max_{y} P(y \mid \text{prompt}; \boldsymbol{\theta})$$

The model maximises probability over its *frozen* parameters $\boldsymbol{\theta}$. There is no external truth signal during inference.

3 Previous Solutions & Their Shortfalls

3.1 A. Fine-Tuning [11:00]

Definition: Fine-Tuning

Adapting a pre-trained LLM by continuing training on a smaller, domain-specific dataset. The model's weights are partially or fully updated so it learns new information.

Training Methods

1. **Supervised Fine-Tuning (SFT):** Labelled pairs (Prompt, Desired Output)
2. **Continued Pre-training:** Unsupervised — feed raw text (e.g., lecture transcripts) directly
3. **RLHF / DPO:** Reinforcement Learning from Human Feedback for alignment

Parameter-Efficient Methods

Method	Description	Cost
Full FT	Updates *all* weights. Most expressive, most expensive.	Very High
LoRA	Low-Rank Adaptation: trains small adapter matrices only.	Low
QLoRA	Quantised LoRA: 4-bit quantisation + LoRA.	Very Low

LoRA Decomposition:

$$W' = W + \Delta W = W + \underbrace{BA}_{\text{low-rank adapter}} \qquad B \in \mathbb{R}^{d\times r}, \quad A \in \mathbb{R}^{r\times k}, \quad r \ll \min(d,\, k)$$

Only A and B are trained. The base weights W remain **frozen**.

Trainable parameters: $r(d+k)$ vs dk for full FT. Typical $r \in \{4, 8, 16\}$.

Fine-Tuning Process (5 Steps)

1. **Collect Data** — thousands of labelled (prompt, output) pairs
2. **Choose Method** — Full FT, LoRA, QLoRA, or other PEFT adapters

3. **Train for Epochs** — freeze base weights (LoRA) or update all (Full FT)
4. **Evaluate & Safety-Test** — exact-match, factuality, hallucination rate; red-team
5. **Handle Hallucinations** — train on "I don't know" examples for tricky prompts

Drawbacks of Fine-Tuning

- **Expensive:** GPU compute cost scales with model size
- **Expertise Required:** Strong ML background needed
- **Stale:** Must retrain every time data changes
- **Knowledge still static:** Same cutoff problem remains

3.2 B. In-Context Learning / Few-Shot Prompting [23:19]

Definition: In-Context Learning

An **emergent property** of very large models (e.g., GPT-3 at 175B params). The model learns a task pattern from *examples placed inside the prompt itself* — no weight updates required.

Reference Paper

"Language Models are Few-Shot Learners" — Brown et al., 2020
https://arxiv.org/pdf/2005.14165

Example Prompt Structure:

```
System: You are a sentiment classifier.
Example 1: Input: "The product is amazing!" Output: Positive
Example 2: Input: "Terrible experience." Output: Negative
Now classify: Input: "It was okay, nothing special." Output: ??? <- Model
infers without retraining
```

Limitation

Prompt size is bounded by the **context window**. You cannot inject thousands of documents. Large corpora cannot fit in a single prompt — hence RAG is needed.

4 What is RAG? [34:03]

Definition: Retrieval-Augmented Generation

RAG combines two AI paradigms to give an LLM access to any external knowledge base at inference time, without modifying its weights.

$$\boxed{\text{RAG} = \underbrace{\text{Information Retrieval}}_{\text{find relevant docs}} + \underbrace{\text{Text Generation}}_{\text{LLM answer}}}$$

Instead of stuffing fixed examples into the prompt (few-shot), RAG **dynamically fetches**

the exact context needed for each individual query.

Core Analogy

Give the LLM the right book to read *before* answering — rather than hoping it memorised the answer during training.

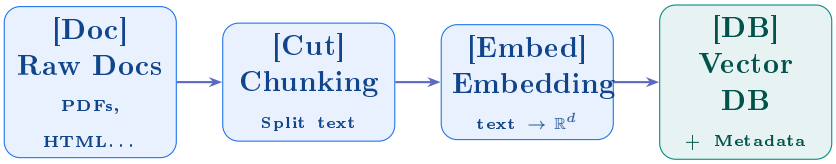
High-Level RAG Flow

5 The 4-Step RAG Architecture [37:36]

5.1 Step 1 — Indexing (Build the Knowledge Base) [41:16]

This happens **offline** / **before** any user query.

Pipeline



Embedding Mathematics

$$\vec{e}_i = f_{\text{embed}}(T_i) \in \mathbb{R}^d$$

The embedding model f maps text chunk T_i to a d-dimensional dense vector.
Typical dimensions: $d \in \{768, 1536, 3072\}$ depending on the model.

Components

Document Sources (LangChain Loaders):

- PDF, DOCX, TXT
- YouTube Transcripts
- AWS S3, Google Drive
- HTML, Markdown, CSV

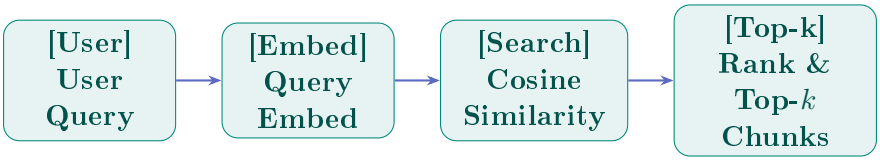
Popular Vector Databases:

- **FAISS** — local, Facebook AI
- **Chroma** — open source
- **Pinecone** — cloud managed
- **Milvus** — distributed

5.2 Step 2 — Retrieval (Semantic Search) [49:13]

This happens **online / at query time**.

Pipeline



Similarity Mathematics

Cosine Similarity:

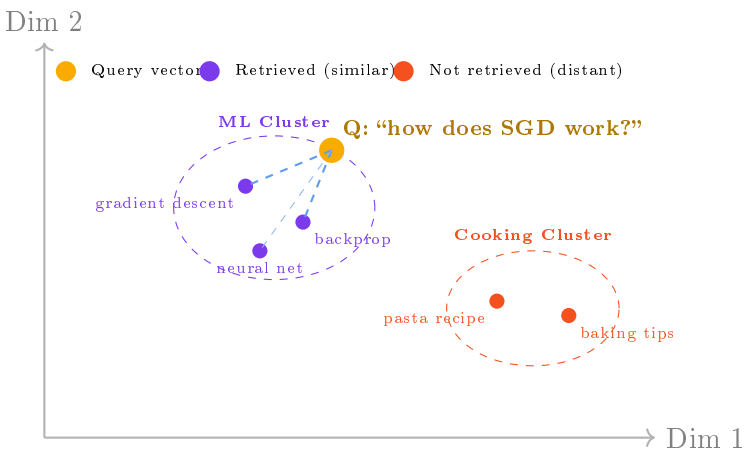
$$\text{sim}(\vec{q},\, \vec{e}_i) \;=\; \frac{\vec{q}\cdot\vec{e}_i}{\|\vec{q}\|\;\|\vec{e}_i\|} \;\in\; [-1,\, 1]$$

Retrieval:

$$\mathcal{C}^* \;=\; \underset{i}{\text{top-}}\,k\;\text{sim}(\vec{q},\, \vec{e}_i)$$

All document vectors $\vec{e}_i$ and query vector $\vec{q}$ live in the **same embedding space**.
Closest vectors $\Rightarrow$ most semantically relevant chunks.

Semantic Embedding Space (Illustration)



5.3 Step 3 — Augmentation [53:07]

Combine the retrieved chunks and user query into a **grounded, context-rich prompt**.

Augmented Prompt Template

```
SYSTEM: You are a helpful assistant.  Answer ONLY using the provided context.
If the answer is not in the context, say "I don't know."
CONTEXT: -- Retrieved Chunk 1 -- [CHUNK 1 TEXT GOES HERE]
-- Retrieved Chunk 2 -- [CHUNK 2 TEXT GOES HERE]
...  (top-k chunks)
USER QUESTION: [USER QUESTION]
ANSWER:
```

5.4 Step 4 — Generation [54:06]

The augmented prompt is sent to the LLM.

Grounded Generation:

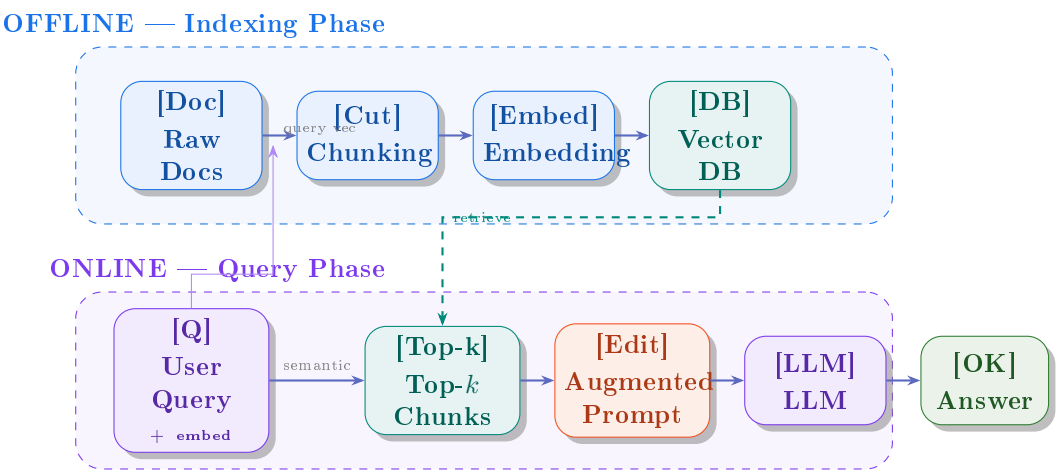
$$\hat{y} = \mathrm{LLM}_{\boldsymbol{\theta}}\left(\underbrace{q}_{\text{query}} \,\Big\|\, \underbrace{C_1, C_2, \ldots, C_k}_{\text{retrieved context}} \right)$$

The model conditions generation on both its internal parameters $\boldsymbol{\theta}$ and the external context chunks C_i.

Grounding Effect

By restricting output to retrieved context, hallucinations drop dramatically — the model cannot "make things up" beyond what it was shown.

Complete RAG Pipeline (Offline + Online)



6 RAG vs Fine-Tuning — Full Comparison [55:36]

Dimension	Fine-Tuning	RAG
Private Data	Baked into weights (static)	✓ Fetched dynamically
Knowledge Cutoff	× Must retrain for new data	✓ Just add new doc to DB
Hallucinations	Can be reduced via training	✓ Grounded in context
Cost	× Very high (GPU hours)	✓ Low (embed + retrieve)
Expertise Needed	× Strong ML background	✓ API-level engineering
Update Frequency	× Expensive retraining	✓ Real-time DB inserts
Task Customisation	✓ Deep style / format tuning	Prompt engineering only
Best For	Style, tone, domain adapt	Knowledge QA, factual retrieval

7 Beyond Naïve RAG — Advanced Techniques

Query Routing

Dynamically decide which vector store or tool to query based on the query type, topic, or intent.

Cross-Encoder Re-ranking

A second-pass model scores each retrieved chunk for *true relevance* and reorders the results before augmentation.

HyDE (Hypothetical Document Embeddings)

Generate a *hypothetical* answer first → embed it → retrieve using that embedding. Dramatically improves recall for complex queries.

Metadata Filtering

Filter the vector store by date, author, source, or category *before* running semantic search to narrow the search space.

8 Quick Revision Summary

* Quick Revision Summary

3 Problems with Standard LLMs:

1. Private Data
2. Knowledge Cutoff
3. Hallucinations

2 Old Fixes (with limits):

1. Fine-Tuning (expensive, stale)
2. Few-Shot Prompting (context limit)

1 RAG Formula:

RAG = Retrieval+Augmentation+Generation

4 RAG Steps:

1. **Index** — Ingest → Chunk → Embed → Store
2. **Retrieve** — Query Embed → Cosine Sim → Top-k
3. **Augment** — System + Context + Query = Prompt
4. **Generate** — LLM → Grounded Answer

Notes based on: Retrieval Augmented Generation | CampusX • Reference: Brown et al., 2020 — Language Models are Few-Shot Learners